

산불 진화자원 배치 알고리즘 연구계획서

연구 주제

산불 진화자원 배치 문제에서 주요 MARL(Multi Agent Reinforcement Learning) 알고리즘의 비교 평가

연구 의의

- 산불 진화 문제에서 MARL 알고리즘 간 비교를 다루는 연구 부족
- 기존 산불 진화 연구에서 MARL은 주로 동일 유형의 자원(예: 동일 UAV 다수)에 한정됨

연구 일정

1주차: wildfire-environment 라이브러리로 시뮬레이션 환경 구축 & heuristic 실험

2주차: MARL 알고리즘 3가지 구현 (MAPPO, MADDPG, MADQN)

3주차: MARL 알고리즘 점검 및 결과 비교

연구 방법

1) 시뮬레이션 환경

Python 오픈 라이브러리인 wildfire-environment를 수정해서 시뮬레이션 환경을 구축할 예정이다. wildfire-environment는 MARLlib 라이브러리를 활용하는 MARL 프로젝트에서 사용하기 위해 개발되었다. 산불 확산(wildfire dynamics)과 다수의 UAV를 이용한 진화 과정을 MDP로 모델링한 OpenAI Gym 기반 multi-agent 시뮬레이션을 제공한다.

wildfire-environment에서 제공하는 기능:

- 격자 기반 산불 시뮬레이션
 - size(격자 크기), initial_fire_size(초기 화점 크기)로 난이도 조절, seed로 재현성 보장(초기 화점 위치 동일)
- multi-agent 운영
 - num_agents로 agent 수 설정, 단일 action space에서 공동 action 처리
- 보상 체계 선택(협력/비협력/그룹)
 - cooperative_reward=True/False로 팀 보상 vs. 개별/그룹 보상 전환
 - 협력 보상에서는 숲 전체 최소 피해 전략, 비협력 보상에서는 각자 담당 구역 우선 전략 성능 비교
- agent별 관심 구역 지정

- `selfish_region_{xmin,xmax,ymin,ymax}`로 agent별 관심 구역 지정, `log_selfish_region_metrics=True`로 구역별 성과 로그 출력
 - 핵심 기반시설이 있는 구역을 더 우선 보호하도록 보상 설계 가능
- 표준 Gym API 호환
 - `reset(seed=...)` → `(obs, info)`, `step(action)` → `(obs, reward, done, info)` 형식 공유
 - RLLib, MARLlib, 자체 PyTorch 루프 어디든 쉽게 연결
- 시각화
 - `render(mode="rgb_array")`로 프레임 추출
- 로그 수집
 - `info` dictionary에 `step` 단위의 진단 지표 제공
- 손쉬운 확장(Wrapper로 전처리/후처리)
 - Gym ObservationWrapper, RewardWrapper, ActionWrapper로 관측값 정규화, 보상 스케일링(환경이 주는 보상값을 그대로 쓰지 않고 곱, clipping 등으로 크기 조정), 매크로액션(여러 `step`을 묶은 고수준 행동) 등 주입 가능

```
import gym
import wildfire_environment

env = gym.make("wildfire-v0",
               num_agents=2,
               size=17,
               initial_fire_size=3,
               cooperative_reward=False,
               log_selfish_region_metrics=True,
               selfish_region_xmin=[7, 13],
               selfish_region_xmax=[9, 15],
               selfish_region_ymin=[7, 1],
               selfish_region_ymax=[9, 3],
               )
observation, info = env.reset(seed=42)

for _ in range(1000):
    action = env.action_space.sample()
    observation, reward, done, info = env.step(action)

    if done:
        observation, info = env.reset()
env.close()
```

그림 1. wildfire_environment를 이용한 환경 생성 예시

본 연구를 위해 수정할 기능:

- agent 종류를 UAV에서 헬리콥터, 소방차량, 인력 3종류로 세분화 (속도, 진화 효율)
 - agent별 사양 예시:
 - helicopter: {speed=2, efficiency=2}
 - truck: {speed=1, efficiency=2}
 - crew: {speed=1, efficiency=1}
 - 방법 1: 코어 수정 없이 wrapper로 구현
 - 속도: 같은 action을 한 step에 여러 번 반복 (k회)
 - 진화 효율: action이 drop(살포)일 때만 추가 drop 반복 (추가 step)

- 방법 2: 코어 수정 (subclassing)
- Cell별 중요도 지정
 - 방법: 가중치 맵 W 을 정의하고(shape = size x size), 매 step마다 피해를 W 로 가중하여 보상을 재정의
 - 현재 그리드 상태로는 `render("rgb_array")` 이용

2) MARL 알고리즘

wildfire-environment로 만든 환경을 이용해 MARLlib 라이브러리로 MARL 모델을 학습시킬 예정이다. 이를 위해 아래 프로젝트 파일 구조에서 wildfire_rl_adapter.py는 wildfire 환경을 RLlib/MARLlib 규격의 멀티에이전트 환경으로의 전환을 담당한다. wrappers.py는 observation, reward, action의 전처리 wrapper를 제공하며, env_config.py은 공통 환경 및 실험 파라미터를 정의한다. run_mappo.py, run_maddpg.py, run_madqn.py는 각 알고리즘의 학습 실행을, evaluate.py는 체크포인트(각 agent의 policy weights 등) 로드, CSV 저장 등을 담당한다.

구현할 알고리즘:

- MAPPO: agent들은 각자 stochastic policy를 가지되, 중앙집중 Critic이 공동 관측을 반영해 학습하는 on-policy 기반 방식 (discrete, continuous action space 모두 잘 됨)
- MADDPG: agent들은 각자 deterministic policy를 가지되, 중앙집중 Critic이 공동 관측 및 다른 모든 agent들의 행동을 반영해 학습하는 off-policy기반 방식 (주로 continuous action space에 효과적)
- MADQN: 중앙집중 Critic 없이 value(Q함수) 기반 방식 (discrete action space 전용)

프로젝트 파일 구조:

```
wildfire_project/
  envs/
    wildfire_rl_adapter.py      # wildfire-env → RLlib MultiAgentEnv 어댑터
    env_config.py              # 공통 환경 설정(size/agents/seed 등)
    wrappers.py                # Observation/Reward/Action wrapper 정의
  scripts/
    run_mappo.py               # 학습 스크립트
    run_maddpg.py
    run_madqn.py
    visualize_live.py          # rgb_array 기반 실시간 시각화
    evaluate.py                # 체크포인트 로드
  runs/
    mappo/ ...                 #로그
    maddpg/ ...
    madqn/ ...
```

3) 평가 방법

본 연구에서 개발한 모델과 비교할 heuristic 후보:

- random: 모든 agent가 무작위 action
- nearest-fire greedy: 각 agent가 가장 가까운 orange(burning) cell로 이동

평가 지표 후보:

- 진화 성공률(%): 종료 시 orange(burning) cell의 수 = 0인 에피소드 비율
- 총 진화 시간(steps): 에피소드 종료까지 걸린 step 수
- 최종 피해 면적(%): 종료 시 brown(소실) cell의 비율
- 가중 피해: cell의 중요도 맵 W 사용 시 $\sum W_{ij} \cdot \text{burned}_{ij}$

참고 문헌

- (1) Tanha, M., Hooshmand, M., Afsharchi, M. (2023). UAV-based Firefighting by Multi-agent Reinforcement Learning. 2023 13th International Conference on Computer and Knowledge Engineering (ICCKE).
- (2) Ramadan, A. (2024). Wildfire Autonomous Response and Prediction Using Cellular Automata (WARP-CA). arXiv preprint arXiv:2407.02613.